软件为何越做越乱？万字长文讲透软件的“复杂性”

原创 营销&交易技术 大淘宝技术 2026年3月16日 02:58 浙江 1489人

本文通过一个电商系统“限时折扣”功能从快速上线到逐渐演变为复杂黑箱的典型案例，深入剖析了软件系统为何越做越乱的根本原因——复杂性的累积。文章指出，复杂性并非源于重大错误，而是由无数看似合理的小妥协（如随意添加开关、复制逻辑、临时补丁）层层叠加所致，最终导致变更放大、认知负荷加重、未知的未知增多。在此基础上，文章对比了“战术性编程”（追求短期交付速度，忽视设计）与“战略性编程”（以长期可维护性为目标，持续投入10%－20%时间优化架构）的本质差异，并系统阐述了管理复杂性的核心方法。作者强调：真正的开发速度来自良好设计，而非侥幸“跑起来”；每一次对设计的轻视，都在为未来的瘫痪埋下伏笔。

01 前言小故事

文章的开始，首先来看一个AI帮忙写的小故事《限时折扣到“补丁风暴”的一年》。

第一周：破局的英雄

电商团队要在两周内上线“限时折扣”。产品经理说，这波活动成败在此。阿杰是团队里的“战术快枪手”，当晚就把需求“跑起来”了：他在订单服务里加了一个大开关，判断当前时间是否命中折扣时段，如果是，就在结算价上直接打一个比例折扣。没有新服务、没有抽象、测试也只有两条冒烟用例。第二天演示一切顺利，大家鼓掌，产品表扬“效率太高”。

第三周：再来一个

“团购”和“会员价”紧接着到来。阿杰把限时折扣的那段逻辑复制了一份，叠加两个if分支，再加上两个Feature Flag。因为赶时间，他把用户等级的查询直接打到了用户服务的同步接口里，并在超时的时候默认用户是普通会员，避免影响下单成功率。上线顺利，数字也漂亮，项目周会上，大家一致认为这套“战术打法”可复制。

第二个月：小问题，但能修

第一个诡异Bug出现：会员价和满减券叠加时，有时多打了一次折。回溯发现是一个分支顺序问题。阿杰十分钟修好了：他在优惠计算前面加了一个短路条件，遇到某些券就跳过后面的逻辑。为稳妥，他又加了一个保护开关，默认开启。产品问是否要重构？阿杰看了看排期表说：“等这波活动过了先，不然这个月目标完不成。”

第四个月：复杂悄悄长大

需求继续滚动：“新人礼金”“跨品类满减”“店铺券优先级”。每次都很紧，每次都能“跑起来”。订单服务里的折扣模块从两个文件长成了十几个文件，方法名里充满了and/or/byGroup/byMember的后缀。构建时间变长了，代码审查也越来越像形式走过场：大家都知道，真正的问题在于设计，但没人有时间停下来改。

第六个月：补丁叠补丁

有一天凌晨，线上报警：东八区和UTC时间换算错了，午夜跨天时部分订单重复计算折扣。阿杰拉起小团队开了个Zoom，二十分钟上线了一个临时补丁：在午夜前后5分钟内，把两个时间都计算一遍，取最小值。问题止住了，技术群里一片赞扬。第二天复盘，大家写下了“待重构”的卡片，又被新一轮活动排期盖过去。

第九个月：谁来重构？

一个关键需求来了：“跨店合并优惠”，而且要支持活动实时生效、规则热更新。阿杰说，这事儿用现在的代码堆不起来了，需要一个规则引擎，把“优惠规则”“应用顺序”“冲突解决”抽成模型，至少两个月。产品摇头：大促在即，必须四周上线。会议室陷入沉默。最终的决定是：先在旧逻辑上面再搭一层“适配器”，把新规则翻译成旧分支能理解的形式。上线勉强过了，但从那之后，任何一个小小的改动都会牵动三处地方，回归测试用例从20条涨到200条。

第十一个月：速度的假象

新来的同学花了一周才敢改第一行逻辑——因为一处看似无害的改动，可能触发四个开关下的三种叠加路径，线上又会冒出某个边角Bug。阿杰依旧很“快”，他熟悉每个旮旯，知道该在哪一层加一个临时判断，问题能被“快刀”切开。团队里的其他人看起来变“慢”了：他们需要读懂上下游的耦合、确认二十个开关的默认状态、跑几百条回归。这种对比让管理层误以为“还是战术有效”。

一年后：代价账单

系统的复杂性不是某个大错误造成的，而是无数小捷径的总和。每一个“合理的妥协”——多一个开关、复制一段逻辑、加一个短路条件——当时都像救命稻草，叠起来就是沼泽。现在，任何一次真正的重构都需要数月时间、跨四个服务、影响五条业务线，排期里根本挤不进去。团队开始习惯了补丁式思维：先把新功能“跑起来”，出了问题再补。补丁带来更多复杂性，更多复杂性产生更多补丁，系统变成了一个谁也不想碰的黑箱。

尾声：如果可以重来

回望最初的“限时折扣”，当时若花两三天把“优惠规则—应用顺序—冲突解决—时间模型”抽出个雏形，设立基本的用例与回归框架，后面每个需求的边际成本都会更低。战术并非不重要，但如果没有战略性设计作为地基，战术的每一次成功，都在为下一次失败埋雷。真正的速度来自良好的设计，而不是一次次“跑起来”的侥幸。

看完上述的小故事，不知道大家是否曾经亲身经历或者遇到过类似的场景。在开发生涯的早期阶段，很多开发者包括我都会犯“战术快枪手”的毛病。战术快枪手总是喜欢“战术编程”，即不为未来做考虑，只想快速地完成眼前的需求，哪怕引入一些复杂性或者bug。而多次的战术编程的代价就是系统变得格外复杂。



复杂性存在多种形式。例如，可能很难理解一段代码是如何工作的。可能需要花费很多精力才能实现较小的改进，或者可能不清楚必须修改系统的哪些部分才能完成新特性。**如果一个软件系统难以理解和修改，那就很复杂。如果很容易理解和修改，那就很简单。**

阅读代码的人比写代码的人更容易理解复杂性。如果你编写了一段代码，可能对你来说很简单，但是其他人则认为它很复杂，那么它就是复杂的。

复杂性的表现

复杂性一般有以下三种表现：

1. **变更放大：**看似简单的变更需要在许多不同地方进行代码修改。
2. **认知负荷：**指开发者需要多少知识才能完成一项任务。较高的认知负担意味着开发者必须花更多的时间来学习所需的信息，并且可能错过重要的东西而导致引发bug的可能性也更大。
3. **未知的未知：**必须修改哪些代码才能完成任务，或者开发者必须获得哪些信息才能成功地执行任务，这些信息不明显。

复杂性的原因

良好设计的最重要目标之一就是使系统显而易见。在一个显而易见的系统中，开发者可以快速了解现有代码的工作方式以及变更所需的内容。一个显而易见的系统是，开发者可以在不费力

地思考的情况下快速猜测要做什么，同时又可以确信该猜测是正确的。

依赖

复杂性的第一个原因是依赖，依赖关系是软件的基本组成部分，无法完全消除。实际上，我们在软件设计过程中有意引入了依赖。每次编写新类时，都会围绕该类的API创建依赖关系。但是，软件设计的目标之一是减少依赖关系的数量，并使依赖关系保持尽可能简单和明显。

晦涩

复杂性的第二个原因是晦涩。当重要的信息不明显时，就会发生模糊。一个简单的例子是变量名称如果过于笼统就没有携带太多有用的信息。或者，某个变量的文档没有说明它的单位（秒还是毫秒？），所以找到它的唯一方法是扫描代码中使用该变量的位置。晦涩常常与依赖项相关联。

依赖和晦涩共同导致了上述描述的三种表现。**依赖导致变化放大和高认知负荷。晦涩会产生未知的未知数，还会增加认知负担**。如果能找到最小化依赖关系和模糊性的设计技巧，那么我们就可以降低软件的复杂性。

复杂性不是由单个致命性错误引起的；而是由很多小的错误堆积而成。之所以会出现复杂性，是因为随着时间的流逝，成千上万的小的依赖和晦涩问题层层堆叠。最终，这些小问题太多了，以至于对系统的每次可能更改都会受到其中几个问题的影响。



前面提到了战术编程，即开发思路是“越快完成任务越好，之后的问题之后再说”。其代码特点是“不会花费太多时间来寻找最佳设计，每次增加一些复杂性或引入一两个小错误”。

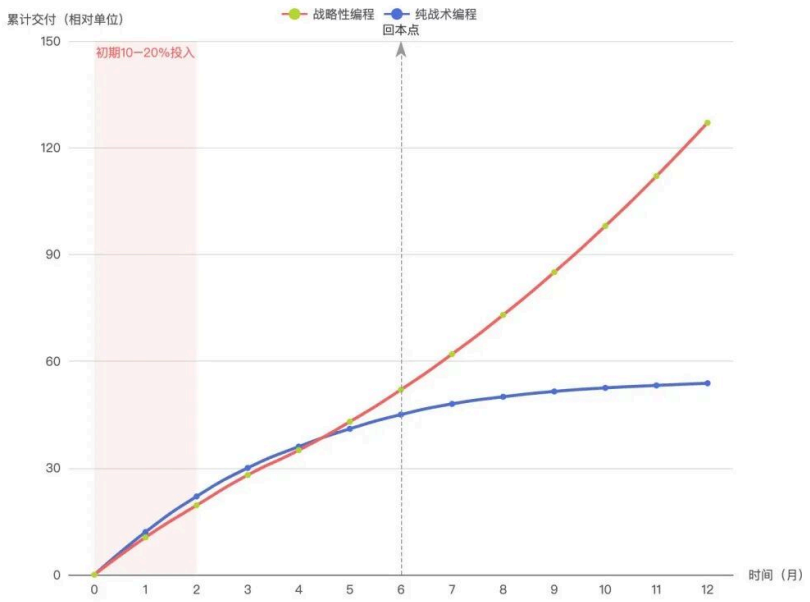
几乎每个软件开发团队都有至少一个将战术编程发挥到极致的开发者：战术快枪手。战术快枪手是一位多产的开发者，他编写代码的速度比其他人快得多。实施新功能时，没有人能比战术快枪手更快地完成任务。在某些组织中，管理层将战术快枪手视为英雄。但是，战术快枪手留下了风卷残云般的痕迹。他们很少被将接手代码的开发者视为英雄。通常，其他开发者必须清理战术快枪手留下的混乱局面，这使得那些接手的开发者的开发速度好像比战术快枪手慢。

成为一名优秀的开发者的第一步是要意识到仅工作代码是不够的。引入不必要的复杂性以更快地完成当前任务不是长宜之计。最重要的是系统的长期架构。任何系统中的大多数代码都是通过扩展现有代码库编写的。因此，作为开发者，最重要的工作就是使得未来的架构易于扩展。

因此，尽管眼前的需求很紧急、很重要，但不应将“完成需求”视为主要目标。我们的主要目标必须是**设计出卓越的架构**，并且这种架构也能满足当前的需求。这是战略。

战略性编程需要一种**投资心态**。我们必须花费时间来改进系统的设计，而不是采取最快的方式来完成当前的项目。这些投资会在短期内让我们放慢脚步，但从长远来看会加快我们的速度。

那么，正确的投资比例是多少？随着我们对系统和业务的不断了解，理想的设计会逐渐出现。因此，最好的方法是连续进行大量小额投资。**建议我们将总开发时间的10％到20％用于投资**。该比例足够小，不会对我们的日常安排产生重大影响，但又足够大，可以随着时间的推移产生重大收益。因此，虽然项目在开始时比纯战术方案多花费10–20％的时间。但是额外的时间将带来更好的软件设计，并且我们将在几个月内开始逐渐体现。不久之后，我们的开发速度将比战术编程快至少10–20％。最终，我们过去投资节省的时间足以抵消一开始做投入的时间。马上就可以收回初始投资的成本。



一种好的思维方式是将今天的事情今天解决。当遇到难题时，大家总是会想下次再去优化。但是，这是一个隐形的坑。一旦这次选择推迟优化，之后也基本会继续推迟优化，并且不知不觉中就变成了战术快枪手。总之，等待解决问题的时间越长，问题就会变得越大；解决方案变得更加令人生畏，这使得轻松推迟解决方案变得更加容易。

04 如何管理复杂性

管理软件复杂性最重要的技术之一就是对系统好好设计，以便开发者在任何规定时间只需要面对整体复杂性的一小部分。

抽象和细节

为了管理依赖关系，我们将每个模块分为两个部分：**接口和实现**。接口包含使用其他模块的开发者必须知道的所有内容，才能使用给定的模块。通常，接口描述模块做什么，而不描述模块如何做。该实现由执行接口所承诺的代码组成。在特定模块中工作的开发者必须了解该模块的接口和实现，以及由给定模块调用的任何其他模块的接口。除了正在使用的模块以外，开发者无需了解其他模块的实现。

最好的模块是那些其接口比其实现简单得多的模块。这样的模块具有两个优点。首先，一个简单的接口可以将模块强加于系统其余部分的复杂性降至最低。其次，如果以不更改其接口的方式修改了一个模块，则该修改不会影响其他模块。如果模块的接口比其实现简单得多，则可以在不影响其他模块的情况下更改模块的许多方面。

模块的接口包含两种信息：**正式信息和非正式信息**。接口的形式部分在代码中明确指定，并且其中一些可以通过编程语言检查其正确性。例如，方法的形式接口是其签名，其中包括其参数的名称和类型，其返回值的类型以及有关该方法引发的异常的信息。

每个接口还包括非正式元素。这些没有以编程语言可以理解或执行的方式指定。接口的非正式部分包括其高级行为，例如，函数删除由其参数之一命名的文件的事实。如果对类的使用在限制（比如必须先调用另一个方法），则这些约束也是类接口的一部分。通常，如果开发者需要了解特定信息才能使用模块，则该信息是模块接口的一部分。接口的非正式方面只能使用注释来描述，而编程语言不能确保描述是完整或准确的。对于大多数接口，非正式方面比正式方面更大，更复杂。

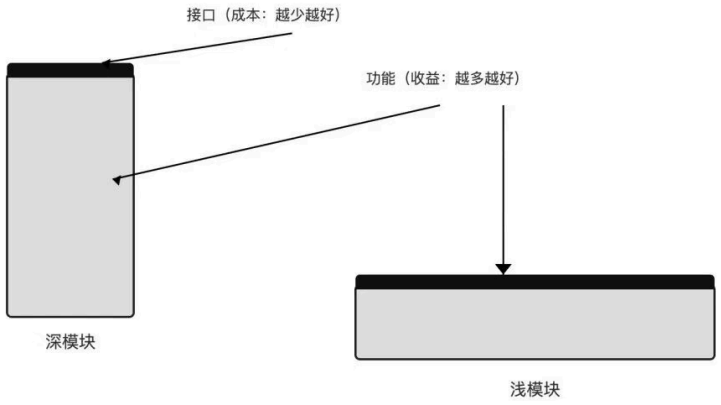
术语抽象与模块化设计的思想紧密相关。抽象是实体的简化视图，其中省略了不重要的细节。抽象是有用的，因为它们使我们更容易思考和操纵复杂的事物。

在抽象的定义中，“无关紧要”一词至关重要。从抽象中忽略的不重要的细节越多越好。但是，如果细节不重要，则只能将其从抽象中省略。**抽象可以通过两种方式出错**。首先，它可以包含**并非真正重要的细节**。当这种情况发生时，它会使抽象变得不必要的复杂，从而增加了使用抽象的开发者的认知负担。第二个错误是抽象**忽略了真正重要的细节**。这导致模糊不清：仅查看抽象的开发者将不会获得正确使用抽象所需的全部信息。忽略重要细节的抽象是错误的抽象：它可能看起来很简单，但实际上并非如此。

我们不仅依靠抽象来管理复杂性，而且不仅在编程中，而且在日常生活中无处不在。举个例子，汽车提供了一种简单的抽象概念，使我们可以在不了解电动机，电池电源管理，防抱死制动，巡航控制等机制的情况下驾驶它们。

深模块和浅模块

最好的模块是那些提供强大功能但具有简单接口的模块。本文用“深入”一词来描述这样的模块。为了形象化深度的概念，假设每个模块都由一个矩形表示。每个矩形的面积与模块实现的功能成比例。矩形的顶部边缘代表模块的接口；边缘的长度表示接口的复杂性。最好的模块很深：它们在简单的接口后隐藏了许多功能。深模块是一个很好的抽象，因为其内部复杂性的一小部分对其用户可见。



模块深度是考虑成本与收益的一种方式。模块提供的好处是其功能。模块的成本是其接口。模块的接口代表了模块强加给系统其余部分的复杂性：接口越小越简单，引入的复杂性就越小。最好的模块是那些收益最大，成本最低的模块。**接口不错，但更多或更大的接口不一定更好**。

深模块的另一个示例是诸如Go或Java之类的语言中的垃圾收集器。这个模块根本没有接口。它在后台进行隐形操作以回收未使用的内存。由于将垃圾收集消除了用于释放对象的接口，因此向系统中添加垃圾回收实际上会缩小其总体接口。垃圾收集器的实现非常复杂，但是使用该语言的开发者却无需关注这种复杂性。

信息隐藏和信息泄露

实现深层模块最重要的技术是信息隐藏。基本思想是每个模块应封装一些知识，这些知识代表设计决策。

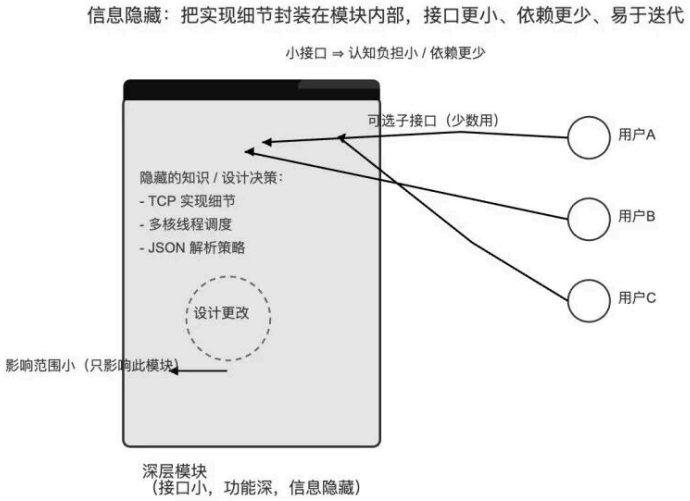
比如如何实现TCP网络协议。如何在多核处理器上调度线程。如何解析JSON文档。这些知识都嵌入在模块的实现中，但不会出现在其接口中，因此其他模块不可见。

- 信息隐藏

信息隐藏在两个方面降低了复杂性。首先，它将接口简化为模块。接口反映了模块功能的更简单、更抽象的视图，并隐藏了细节;**这减少了使用该模块的开发者的认知负担**。其次，**信息隐藏使系统更容易迭代**。如果隐藏了一段信息，那么在包含该信息的模块之外就不再对该信息的依赖，因此与该信息相关的设计更改将只影响一个模块。设计新模块时，应仔细考虑可以在该模块中隐藏哪些信息。如果我们可以隐藏更多信息，则还应该能够简化模块的接口，这会使模块更深。

信息隐藏的最佳形式是将信息完全隐藏在模块中，从而使该信息对模块的用户无关且不可见。但是，部分信息隐藏也具有价值。例如，如果某个类的某些用户仅需要特定的功能或信息，并且可以通过单独的方法对其进行访问，以使其在最常见的用例中不可见，则该信息通常会被隐藏。与类的每个用户可见的信息相比，此类信息将创建更少的依赖项。

除此之外，信息隐藏也可以应用于系统中的其他级别，例如类级别。尝试在一个类中设计私有方法，以便每个方法都封装一些信息或功能，并将其隐藏在类的其余部分中。此外，请尽量减少使用每个实例变量的位置数量。有些变量可能需要在整个类中广泛使用，但是其他变量可能只需要在少数地方使用；如果可以减少使用变量的位置的数量，则将消除类内的依赖关系并降低其复杂性。



- 信息泄漏

信息隐藏的反面是信息泄漏。当一个设计决策反映在多个模块中时，就会发生信息泄漏。这在模块之间创建了依赖关系：对该设计决策的任何更改都将要求对所有涉及的模块进行更改。**如果格式更改，则两个类都将需要修改**。

信息泄漏是软件设计中最重要的危险信号之一。作为一个开发者，我们能学到的最好的技能之一就是对信息泄露的高度敏感性。如果我们在类之间发现信息泄漏，请自问“**我如何才能重新组织这些类，使这些特定的知识只影响一个类**?”如果受影响的类相对较小，并且与泄漏的信息紧密相关，那么将它们合并到一个类中是有意义的。另一种可能的方法是从所有受影响的类中提取信息，并创建一个只封装这些信息的新类。但是，这种方法只有在我们能够找到一个从细节中抽象出来的简单接口时才有效;如果新类通过其接口公开了大部分知识，那么它就不会提供太多的价值。

信息泄漏的一个常见原因称为**时间分解**的设计风格。在时间分解中，系统的结构对应于操作将发生的时间顺序。考虑一个应用程序，该应用程序以特定格式读取文件，修改文件内容，然后再次将文件写出。通过时间分解，该应用程序可以分为三类：一类用于读取文件，另一类用于执行修改，第三类用于写出新版本。文件读取和文件写入步骤都具有有关文件格式的知识，这会导致信息泄漏。解决方案是将用于读写文件的核心机制结合到一个类中。该类将在应用程序的读取和写入阶段使用。在日常开发过程中，我们很容易陷入时间分解的陷阱，因为在编写代码时通常会想到必须执行操作的顺序。但是，大多数设计决策会在应用程序的整个生命周期中的多个不同时刻表现出来。结果就是时间分解常常导致信息泄漏。

在时间分解中，执行顺序反映在代码结构中：在不同时间发生的操作在不同的方法或类中。如果在执行的不同点使用相同的知识，则会在多个位置对其进行编码，从而导致信息泄漏。

最后需要提醒的是，仅当在其模块外部不需要隐藏信息时，隐藏信息才有意义。如果模块外部需要该信息，则不得隐藏它。假设模块的性能受某些配置参数的影响，并且模块的不同用途将需要对参数进行不同的设置。在这种情况下，将参数暴露在模块的接口中很重要，以便可以对其进行适当的旋转。作为开发者，我们的目标应该是最大限度地减少模块外部所需的信息量。例如，如果模块可以自动调整其配置，那将比公开配置参数更好。但是，最重要的是要识别模块外部需要哪些信息，并确保将其公开。



设计新模块时，我们面临的最普通的决策之一就是是**以通用还是专用方式实现它**。有人可能会说，不管什么情况下，我们都应该采用通用方法，我们应该追求一种可用于解决广泛问题的机制，而不仅是眼前重要的问题。在这种情况下，新机制可能会在将来发现意外用途，从而节省时间。

另一方面，我们知道很难预测软件系统的演变方向，因此通用解决方案可能包含从未真正需要的功能。此外，**如果我们实现的东西过于通用，那么可能无法很好地解决我们今天遇到的特定**

问题。结果，另外一些人可能会说，最好只关注眼前的需求，完成我们所知道的需求，并针对我们今天打算使用的方式进行特殊处理。如果我们采用特殊处理并在以后发现更多用途，则始终可以对其进行重构以使其通用。专用方法似乎与软件开发的增量方法一致。

“有点”通用比较好

以笔者个人的经验，最有效的方法是有点通用的方式实现新模块。“**有点通用**”表示该模块的功能应反映我们当前的需求，但其接口则不应该这样。相反，该接口应该足够通用以支持多种用途。该接口应易于使用，以满足眼前的需求，而不必专门与它们联系在一起。“有点”这个词很重要：不要被带偏追求过于通用的东西，以至于反而很难满足当前的需求。

通用方法最重要的好处是，与专用方法相比，它产生了更简单，更深入的接口。如果我们将该类用于其他目的，则通用方法还可以节省将来的时间。但是，即使该模块仅用于其原始用途，由于其简单性，通用方法仍然更好。

满足我当前所有需求的最简单的接口是什么？

如果在不减少 API 整体功能的情况下减少其方法数量，那么很可能是创建了更加通用的方法。只有在每个方法的 API 保持简单的情况下，减少方法的数量才有意义。如果为了减少方法的数量而不得不引入大量额外的参数，那可能这不是真正的简化。

这个接口会在多少情况下使用？

如果一个方法是为专用而设计的，则意味着它可能过于专用。看看能否用一个通用方法取代几个专用方法。

这个接口是否已经满足我当前的需求？

这个问题可以帮助我们确定何时使接口变得简单而通用。如果我们必须编写许多其他代码才能将类用于当前用途，那么这是一个危险信号，即该接口未提供正确的功能。

通用接口比专用接口具有许多优点。它们往往更简单，使用的方法更少。它们还提供了类之间的更清晰的分隔，而专用接口则倾向于在类之间泄漏信息。使模块具有某种通用性是降低整体系统复杂性的最佳方法之一。

专用代码无法完全避免

但是大多数软件系统都不可避免地有一些专用代码。例如应用程序为其用户提供专用特性，这些特性通常是高度专用的。因此通常不可能完全消除专用性。不过专用代码应该与通用代码清晰的分离，这可以通过将专用代码在软件栈中向上推或者向向下推来实现。

分离专用代码的一种方法是向上推。应用程序的顶层类提供专用特性，必然会针对这些专用特性做一些优化。但这种专用并不会渗透到用于实现这些特性的底层类中。

有时最好的方法是向下推专用代码。设备驱动程序就是一个例子。操作系统通常必须支持成百上千种不同的设备类型。设备驱动程序模块使得使用该设备的专用特性来实现通用接口。这种方法将专用代码推向了设备驱动程序。因此在编写操作系统核心时，无需了解特定设备的特性。这种方法也让添加新设备变得更加容易。如果一个设备有足够的特性来实现设备驱动程序接口，就可以将其添加到系统中，而无需对主操作系统进行任何修改。



软件系统由层组成，其中较高的层使用较低层提供的功能。在设计良好的系统中，每一层都提供与其上、下两层不同的抽象。如果我们通过调用方法遵循单个操作在层中上下移动，则每个方法调用的抽象都会改变。如果系统包含具有相似抽象的相邻层，则这是一个危险信号，表明类分解存在问题。
下面讲述一些在分层抽象中我们当遇到的一些设计问题。

直通方法

当相邻的层具有相似的抽象时，问题通常以直通方法的形式表现出来。直通方法是一种很少执行的方法，除了调用另一个方法（其签名与调用方法的签名相似或相同）之外。

```
@Repository
@Transactional(readOnly = true)
public class SimpleJpaRepository<T, ID> implements JpaRepositoryImplem

    // 直通方法：几乎直接委托给EntityManager
    @Override
    public List<T> findAll() {
        return getQuery(null, Sort.unsorted()).getResultList();
    }

    @Override
    public List<T> findAll(Sort sort) {
        return getQuery(null, sort).getResultList();
    }

    @Override
```

```
17      public Page<T> findAll(Pageable pageable) {
18          // 直通方法：只是添加分页逻辑后委托
19          if (isUnpaged(pageable)) {
20              return new PageImpl<>(findAll());
21          }
22          return findAll(pageable, null);
23      }
24
25      @Override
26      public Optional<T> findById(ID id) {
27          Assert.notNull(id, ID_MUST_NOT_BE_NULL);
28          Class<T> domainType = getDomainClass();
29
30          if (metadata == null) {
31              // 直通方法：直接委托给EntityManager.find()
32              return Optional.ofNullable(em.find(domainType, id));
33          }
34          // 稍微复杂一点但本质还是直通
35          LockModeType type = metadata.getLockModeType();
36          Map<String, Object> hints = getQueryHints().withFetchGraphs(em
37          return Optional.ofNullable(type == null ? em.find(domainType,
38                                      em.find(domainType, id, type, hints))
39      }
40
41      @Transactional
42      @Override
43      public void deleteById(ID id) {
44          Assert.notNull(id, ID_MUST_NOT_BE_NULL);
45          // 直通方法：查找后删除
46          delete(findById(id).orElseThrow(() -> new EmptyResultDataAcces
47                  String.format("No %s entity with id %s exists!", entit
48      }
49
50      @Transactional
51      @Override
52      public void delete(T entity) {
53          Assert.notNull(entity, "Entity must not be null!");
54          // 直通方法：直接委托给EntityManager
55          if (entityInformation.isNew(entity)) {
56              return;
57          }
58          Class<?> type = ProxyUtils.getUserClass(entity);
59          T existing = (T) em.find(type, entityInformation.getId(entity)
60          if (existing == null) {
61              return;
62          }
```

```
        em.remove(em.contains(entity) ? entity : em.merge(entity));
    }
}

```

直通方法使类变浅：它们增加了类的接口复杂性，从而增加了复杂性，但是并没有增加系统的整体功能。在上述CRUD相关的方法中，前面三个基本没有什么功能。直通方法还会在类之间创建依赖关系。

但是也不能一棒子打死直通方法，**我们需要警惕的是尽量不要在一个类内，大部分都是直通方法**。可以因为代码分层小部分是直通方法。如果真的遇到了一个类里大部分都是直通方法。那解决方案可以是重构这个类，以使每个类都有各自不同且连贯的职责。比如能不能把直通对应的关联类合并为一个类，或者索性可以让更上层直接依赖直通关联的两个类。

另外还需要警惕的就是**滥用装饰器设计模式**，装饰器的动机是将类的专用扩展与更通用的核心分开。但是，装饰器类往往很浅：它们引入了大量的样板，以实现少量的新功能。装饰器类通常包含许多直通方法。过度使用装饰器模式很容易，为每个小的新功能创建一个新类，但是会导致浅层类激增。

以`org.springframework.data.repository.support.RepositoryComposition`类举个例子：

```
public class RepositoryComposition {

    // 装饰器组合类，主要是样板代码
    private final Fragments fragments;

    private RepositoryComposition(Fragments fragments) {
        this.fragments = fragments;
    }

    public static RepositoryComposition empty() {
        return new RepositoryComposition(Fragments.empty());
    }

    public static RepositoryComposition just(RepositoryFragment fragme
        return new RepositoryComposition(Fragments.just(fragment));
    }

    // 直通方法
    public RepositoryComposition append(RepositoryFragment fragment) {
        return new RepositoryComposition(this.fragments.append(fragmen
```

```
    }

    public RepositoryComposition prepend(RepositoryFragment fragment)
        return new RepositoryComposition(this.fragments.prepend(fragme
    }

    // 更多直通方法
    public Streamable<RepositoryFragment> getFragments() {
        return this.fragments;
    }

    public RepositoryFragment getFragment(Class<?> interfaceClass) {
        return this.fragments.getFragment(interfaceClass);
    }

    // 浅层装饰器的典型特征
    static class Fragments implements Streamable<RepositoryFragment> {

        private final List<RepositoryFragment> fragments;

        private Fragments(List<RepositoryFragment> fragments) {
            this.fragments = fragments;
        }

        static Fragments empty() {
            return new Fragments(Collections.emptyList());
        }

        static Fragments just(RepositoryFragment fragment) {
            return new Fragments(Collections.singletonList(fragment));
        }

        // 大量样板装饰方法
        Fragments append(RepositoryFragment fragment) {
            List<RepositoryFragment> newFragments = new ArrayList<>(th
            newFragments.addAll(this.fragments);
            newFragments.add(fragment);
            return new Fragments(newFragments);
        }

        Fragments prepend(RepositoryFragment fragment) {
            List<RepositoryFragment> newFragments = new ArrayList<>(th
            newFragments.add(fragment);
            newFragments.addAll(this.fragments);
            return new Fragments(newFragments);
        }
```

```
    }
}

```

传递变量

传递变量是通过一长串方法向下传递的变量。传递变量增加了复杂性，因为它强制所有中间方法知道它们的存在，即使这些方法对变量没有用处。此外，如果存在一个新变量，则可能必须修改大量的接口和方法才能将变量传递给所有相关路径。以Apache Struts 1.x的ActionContext为例，可以看到从起始execute方法到最终的checkUserInput，一路上所有方法都必须包含mapping, form, request, response四个变量。

```
public class UserAction extends Action {

    public ActionForward execute(ActionMapping mapping, ActionForm for
                                 HttpServletRequest request, HttpServlet

        return processUserAction(mapping, form, request, response);
    }

    private ActionForward processUserAction(ActionMapping mapping, Act
                                            HttpServletRequest request,

        validateInput(mapping, form, request, response);
        return performBusinessLogic(mapping, form, request, response);
    }

    private void validateInput(ActionMapping mapping, ActionForm form,
                               HttpServletRequest request, HttpServletRe
        // 可能只需要form和request，但必须传递所有参数
        checkUserInput(mapping, form, request, response);
    }

    private void checkUserInput(ActionMapping mapping, ActionForm form
                                HttpServletRequest request, HttpServletR
        // mapping和response在这里可能不需要，但必须传递
        String username = ((LoginForm) form).getUsername();
        // 验证逻辑...
    }
}

```

但实际情况中，由于代码分层或业务逻辑的问题，又不可避免会存在从顶端到底层的变量，完全消除传递变量是不可能的，只能相对性地做一些优化。以下是几种解决方案：

- 查看最顶层和最底层方法之间是否已共享对象。
- 将信息存储在全局变量中，如果是串行处理的话也可以考虑用threadlocal（但threadlocal具有隐蔽性）。
- 引入一个上下文对象（context）。上下文对象专门从来存储所有全局状态。大多数应用程序在处理时都需要共享全局状态下具有多个变量，比如配置项，计数器之类的内容。

笔者现在最常用的解决方案是上下文对象方案，**上下文对象的优点是统一了所有系统全局信息的处理，并且不需要传递变量。如果需要添加新变量，则可以将其添加到上下文对象**；除了上下文的构造函数和析构函数外，现有代码均不受影响。由于上下文全部存储在一个位置，因此上下文可以轻松识别和管理系统的全局状态。上下文也便于测试：测试代码可以通过修改上下文中的字段来更改应用程序的全局配置。如果系统使用传递变量，则实施此类更改将更加困难。

上下文远非理想的解决方案。存储在上下文中的变量具有全局变量的大多数缺点。例如，哪些特定变量用在哪里的很不明显。与此同时，上下文容易逐步变成巨大的「大泥球」，从而在整个系统中创建不明显的依赖关系。上下文也可能产生线程安全问题（解决方法是让上下文中的一些变量不可变）。但是在较为复杂的业务场景下，上下文方案已经算是较好的解决方案了。

07 简单留给别人，复杂交给自己

假设我们正在开发一个新模块，并且发现存在不可避免的复杂性。那这个复杂性到底由谁承担更好：应该让模块使用者处理复杂性，还是应该在模块开发者处理复杂性？如果复杂度与模块提供的功能有关，那大多数情况下应该由模块开发者处理。因为大多数情况下，模块使用者多于开发者，因此开发者麻烦些总是比使用者麻烦些好。作为模块开发者，应该努力使模块使用者的开发尽可能轻松，即使这意味着开发者需要额外的工作。换句话说，**模块具有简单的接口比简单的实现更为重要**。

但是实际开发中，作为开发者却很容易做相反的事：把简单留给自己，把复杂交给别人。比如如果出现不确定如何处理的边界条件，最简单的方法是引发异常并让调用方处理它；再如果不确定要实施什么策略，就定义一些配置参数来控制该策略，然后由调用方自行确定最佳策略。这样的方法虽然短期内让模块的开发者轻松，但它们会增加复杂性，因为调用方必须处理这个被「抛」出来的问题（而且往往不止一个调用方）。例如，如果一个类抛出异常，则该类的每个调用者都必须处理该异常。如果一个类导出配置参数，则每个使用这个类的调用方都必须学习如何正确使用它们。

配置参数

配置参数是提高复杂度而不是降低复杂度的一个示例。类可以在内部输出一些控制其行为的参数，而不是在内部确定特定的行为，例如失败重试次数。然后，该类的用户必须为参数指定适当的值。在现在的系统中，配置参数非常流行，到处可见。但是，配置参数并不是一种很好的处理方式，它虽然提供了一定的灵活性。但是在许多情况下，调用方很难或无法确定参数的正确值。

如果可能的话，应该尽可能在系统内部通过一些额外的工作来自动确定正确的值。以网络丢包问题举例。如果发送了请求但在一定时间内未收到响应，则需要重新发送该请求。确定重试间隔的一种方法是引入配置参数。但是，传输协议可以通过测量成功请求的响应时间，自己计算出一个合理的值。这种方法降低了复杂性，使用户不必自己找出正确的重试间隔。相反，如果让调用方配置参数，由于不知道网络协议细节，容易无法计算出合适的值。

因此，应尽可能避免使用配置参数。在使用配置参数之前，可以先问自己一个问题："用户是否能够确定比我们在此确定的更好的值？"当我们创建配置参数时，请查看是否可以自动计算合理的默认值，因此用户仅需在特殊情况下提供值即可。理想情况下，每个模块都应完全自主解决问题。配置参数导致解决方案不完整，从而增加了系统复杂性。

08 在一起更好还是分开更好

软件设计中最基本的问题之一是：给定两个功能，它们应该在同一位置一起实现，还是应该分开实现？这个问题基本适用于系统中的所有级别，例如功能，方法，类和服务。

在决定是合并还是分开时，目标是降低整个系统的复杂性并改善其模块化。过往的经验总是推荐将系统划分为大量的小组件：组件越小，每个单独的组件可能越简单。但是，**细分的行为也会带来额外的复杂性**，而这在细分之前是不在的。

- **组件越多，就越难以追踪所有组件**：也就越难在大型代码接口中找到所需的组件。细分通常会导致更多接口，并且每个新接口都会增加复杂性。
- **细分可能会导致附加代码来管理组件**：例如，在细分之前使用单个对象的一段代码现在可能必须管理多个对象。
- **细分产生分离**：细分后的组件将比细分前的组件距离更远。例如，在细分之前位于单个类中的方法可能在细分之后位于不同的类中。分离使开发者更难同时查看这些组件，甚至不知道它们的存在。如果组件真正独立，那么分离是好的：它使开发者可以一次专注于单个

组件，而不会被其他组件分散注意力。但是如果组件之间存在依赖，则分离是不好的：开发者最终将在组件之间来回跳转。更糟糕的是，他们可能不了解依赖关系，导致理解错误。

- **细分导致重复**：细分之前的单个实例中存在的代码可能需要存在于每个细分的组件中。

那么我们该如何判断应该独立还是分离呢？以下是判断标准：

- 是否共享信息；例如，这两段代码都可能取决于输入的文件格式（Excel还是PDF之类的）。
- 是否一起使用：任何使用A代码的人都可能同时使用B代码。作为反例，磁盘块高速缓存几乎总是包含哈希表，但是哈希表可以在许多不涉及块高速缓存的情况下使用，因此模块应该分开。
- 是否概念上重叠：是否存在一个更高级别的类，会同时包括这两段代码。例如，搜索子字符串和大小写转换都属于字符串操作类。
- 是否不看A的代码就很难理解B的代码。

通用还是专用

通常来说，在设计理念上，**系统的下层倾向于更通用，而上层则更专用**。例如，应用程序的最顶层包含完全特定于该应用程序的功能。将专用代码与通用代码分开的方法是将专用代码向上拉到较高的层，而将较低的层保留为通用。当我们遇到同时包含通用功能和专用功能的类时，请查看该类是否可以分为两个类，一个包含通用功能，另一个在其上层提供特殊功能。

拆分还是合并

何时细分的问题不仅适用于类，而且还适用于方法：是否有时最好将现有方法分为多个较小的方法？还是应该将两种较小的方法合并为一种较大的方法？长方法比短方法更难于理解，因此许多人认为仅长度是分解方法的一个很好的理由。课堂上的学生通常会获得严格的标准，例如"拆分超过20行的任何方法！"

但是，长度本身并不是拆分方法判断条件的一个很好的理由。通常，开发人员倾向于过多地分解方法。拆分方法会引入其他接口，从而增加了复杂性。它还将原始方法的各个部分分开，如果这些部分实际上是相关的，则使代码更难阅读。**长方法并不总是坏的**。以 `org.springframework.beans.factory.support.AbstractBeanFactory` 中的 `doGetBean()` 举例，这是Spring框架中的长方法之一，大约160行代码。

```
protected <T> T doGetBean(
```

```
                        String name, @Nullable Class<T> requiredType,
throws BeansException {

String beanName = transformedBeanName(name);
                Object bean;

// Eagerly check singleton cache for manually registered singletons.
Object sharedInstance = getSingleton(beanName);
if (sharedInstance != null && args == null) {
if (logger.isTraceEnabled()) {
if (isSingletonCurrentlyInCreation(beanName)) {
                                        logger.trace("Returning eager
"' that is not fully initialized yet - a consequence of a circular re
                                }
else {
                                        logger.trace("Returning cache
                                }
                        }
                        bean = getObjectForBeanInstance(sharedInstanc
                }

else {
// Fail if we're already creating this bean instance:
// We're assumably within a circular reference.
if (isPrototypeCurrentlyInCreation(beanName)) {
throw new BeanCurrentlyInCreationException(beanName);
                        }

// Check if bean definition exists in this factory.
BeanFactory parentBeanFactory = getParentBeanFactory();
if (parentBeanFactory != null && !containsBeanDefinition(beanName)) {
// Not found -> check parent.
String nameToLookup = originalBeanName(name);
if (parentBeanFactory instanceof AbstractBeanFactory) {
return ((AbstractBeanFactory) parentBeanFactory).doGetBean(
                                                        nameToLookup,
                                }
else if (args != null) {
// Delegation to parent with explicit args.
return (T) parentBeanFactory.getBean(nameToLookup, args);
                                }
else if (requiredType != null) {
// No args -> delegate to standard getBean method.
return parentBeanFactory.getBean(nameToLookup, requiredType);
                                }
else {
```

```
48 return (T) parentBeanFactory.getBean(nameToLookup);
49                                 }
50                         }
51
52 if (!typeCheckOnly) {
53                                 markBeanAsCreated(beanName);
54                         }
55
56 try {
57 RootBeanDefinition mbd = getMergedLocalBeanDefinition(beanName);
58                                 checkMergedBeanDefinition(mbd, beanNa
59
60 // Guarantee initialization of beans that the current bean depends on
61                                 String[] dependsOn = mbd.getDependsOn
62 if (dependsOn != null) {
63 for (String dep : dependsOn) {
64 if (isDependent(beanName, dep)) {
65 throw new BeanCreationException(mbd.getResourceDescription(), beanNam
66 "Circular depends-on relationship between '" + beanName + "' and '" +
67                                                         }
68                                                         registerDependentBean
69 try {
70                                                                 getBean(dep);
71                                                         }
72 catch (NoSuchBeanDefinitionException ex) {
73 throw new BeanCreationException(mbd.getResourceDescription(), beanNam
74 "'" + beanName + "' depends on missing bean '" + dep + "'", ex);
75                                                         }
76                                         }
77                                 }
78
79 // Create bean instance.
80 if (mbd.isSingleton()) {
81                                         sharedInstance = getSingleton
82 try {
83 return createBean(beanName, mbd, args);
84                                                         }
85 catch (BeansException ex) {
86 // Explicitly remove instance from singleton cache: It might have bee
87 // eagerly by the creation process, to allow for circular reference r
88 // Also remove any beans that received a temporary reference to the b
89                                                                 destroySingle
90 throw ex;
91                                                         }
92                                         });
93                                         bean = getObjectForBeanInstan
```

```
94                                  }
95
96  else if (mbd.isPrototype()) {
97  // It's a prototype -> create a new instance.
98  Object prototypeInstance = null;
99  try {
100                                                 beforePrototypeCreati
101                                                 prototypeInstance = c
102                                             }
103 finally {
104                                                 afterPrototypeCreatio
105                                             }
106                                             bean = getObjectForBeanInstan
107                                 }
108
109 else {
110 String scopeName = mbd.getScope();
111 if (!StringUtils.hasLength(scopeName)) {
112 throw new IllegalStateException("No scope name defined for bean ´" + b
113                                             }
114 Scope scope = this.scopes.get(scopeName);
115 if (scope == null) {
116 throw new IllegalStateException("No Scope registered for scope name ''
117                                             }
118 try {
119 Object scopedInstance = scope.get(beanName, () -> {
120                                                                 beforePrototyp
121 try {
122 return createBean(beanName, mbd, args);
123                                                                 }
124 finally {
125                                                                         afterP
126                                                                 }
127                                                 });
128                                                 bean = getObjectForBea
129                                             }
130 catch (IllegalStateException ex) {
131 throw new BeanCreationException(beanName,
132 "Scope '" + scopeName + "' is not active for the current thread; consi
133 "defining a scoped proxy for this bean if you intend to refer to it f
134                                                                         ex);
135                                             }
136                                 }
137                         }
138 catch (BeansException ex) {
139                                 cleanupAfterBeanCreationFailure(beanNa
```

```
140 throw ex;
141                         }
142                 }
143
144 // Check if required type matches the type of the actual bean instanc
145 if (requiredType != null && !requiredType.isInstance(bean)) {
146 try {
147 T convertedBean = getTypeConverter().convertIfNecessary(bean, require
148 if (convertedBean == null) {
149 throw new BeanNotOfRequiredTypeException(name, requiredType, bean.get
150                                 }
151 return convertedBean;
152                         }
153 catch (TypeMismatchException ex) {
154 if (logger.isTraceEnabled()) {
155                                         logger.trace("Failed to conve
156                                                         ClassUtils.ge
157                                 }
158 throw new BeanNotOfRequiredTypeException(name, requiredType, bean.get
159                         }
160                 }
161 return (T) bean;
162         }
```

以上代码块相对独立，所以可以一次阅读所有逻辑。如何将其中的代码块抽到单独的方法中并没有太大的好处。如果这些代码块具有复杂的交互作用，则将它们保持在一起就显得尤为重要，这样阅读代码的人就可以一次看到所有代码。如果每个块使用单独的方法，则阅读代码的人将不得不在这些扩展方法之间来回切换，以了解它们如何协同工作。如果方法具有简单的签名并且易于阅读，则包含数百行代码的方法就可以了。这些方法很深就没关系。

总之，设计方法时，最重要的目标是提供简洁的抽象。每种方法都应该做一件事并且完全做到这一点。该方法应该具有简洁的接口，以便用户无需费神就可以正确使用它。该方法应该很深：其接口应该比其实现简单得多。如果一个方法具有所有这些属性，那么它的长短与否可能无关紧要。



简化异常处理逻辑

异常处理是软件系统中最糟糕的复杂性来源之一。处理特殊情况的代码在本质上比处理正常情况的代码更难编写，并且开发者经常在定义异常时不考虑异常的处理方式。

许多编程语言都包含一种正式的异常机制（Java的try-catch，Python的try-excep等），该机制允许底层代码抛出异常，由外层代码捕获处理。但是，即使不使用正式的异常机制，也可能发生异常，比如某个方法返回一个特殊值指示其未完成其正常行为时（比如redis的set带条件参数时返回nil表示失败）。所有这些形式的异常都会增加复杂性。以下几种情况都可能造成异常：

- **参数错误**：调用方可能会提供错误的参数或配置信息。
- **资源异常**：调用的方法可能无法完成请求的操作。例如，I/O 操作可能失败，或者所需的资源可能不可用。
- **分布式系统网络异常**：在分布式系统中，网络数据包可能会丢失或延迟，服务器可能无法及时响应。
- **系统内部异常**：该代码可能会检测到错误，内部不一致或未准备处理的情况。

异常处理代码天生就比正常情况下的代码更难写。异常中断了正常的代码流；它通常意味着某事没有像预期的那样工作。当异常发生时，开发者可以用两种方法处理它，每种方法都很复杂。**第一种方法是向前推进并完成正在进行的工作**。例如，如果一个网络数据包丢失，它可以被重发；如果数据损坏了，也许可以从冗余副本中恢复数据。**第二种方法是中止正在进行的操作，向上报告异常**。但是，中止可能很复杂，因为异常可能发生在系统状态不一致的地方；异常处理代码必须处理一致性问题，例如撤销发生异常之前所做的任何更改。

此外，**异常处理代码还可能造成更多异常**。考虑重新发送丢失的网络数据包的情况。也许该数据包实际上并没有丢失，但是只是被延迟了。在这种情况下，重新发送数据包将导致重复的数据包到达接收方；这引入了接收方必须处理的新异常情况。或者，考虑从冗余副本恢复丢失的数据的情况：如果冗余副本也丢失了怎么办？在恢复期间发生的次要异常通常比主要异常更加微妙和复杂。如果通过中止正在进行的操作来处理异常，则必须将此异常作为另一个异常报告给调用方。为了防止无休止的异常级联，开发者最终必须找到一种在不引入更多异常的情况下处理异常的方法。

确保异常处理代码真正起作用是困难的。某些异常（例如I/O错误）在测试环境中不易生成，因此很难测试处理它们的代码。异常在运行的系统中很少发生，因此异常处理代码很少执行。错误可能会长时间未被发现，并且当最终需要异常处理代码时，它很有可能无法正常工作。

异常是不是越多越好

开发者通过定义不必要的异常加剧了与异常处理有关的问题。绝大多数开发者都被教导异常处理很重要。他们通常将其解释为“检测到的错误越多越好”。这导致了一种过分防御的风格，其

中任何看起来甚至有点可疑的东西都被拒绝，并带有异常，这导致了不必要的异常的泛滥，从而增加了系统的复杂性。

直接抛出异常的解决方案听起来很诱人：与其想出一种干净的方法来处理它，不如抛出一个异常并将问题转交给调用者。有人可能会争辩说，这种方法可以赋予调用者权力，因为它允许每个调用者以不同的方式处理异常。但是，**如果开发者在确定情况下都不知道该怎么做，那么怎么能期待调用方能知道怎么做呢**。这种情况下生成的异常只会将问题传递给其他人，并增加系统的复杂性。

抛出异常很容易；处理它们很困难。因此，异常的复杂性来自异常处理代码。减少由异常处理引起的复杂性破坏的最佳方法是减少必须处理异常的数量。

尽可能少定义异常

消除异常处理复杂性的最好方法是定义我们的API，提供更多默认行为，减少抛出异常的代码。这看起来有点奇怪，但在实践中非常有效。

以 Spring 中的 `AbstractApplicationContext` 的异常处理策略举例，由于实现AutoCloseable接口了close()方法，就算在refresh过程中出现了异常行为，也可以正确的清理资源，而不是将异常抛给调用者，并且让调用者处理异常带来的烂摊子。

```
public abstract class AbstractApplicationContext extends DefaultResour
        implements ConfigurableApplicationContext {

    @Override
    public void refresh() throws BeansException, IllegalStateException
        synchronized (this.startupShutdownMonitor) {
            prepareRefresh();

            try {
                // 核心刷新逻辑
                ConfigurableListableBeanFactory beanFactory = obtainFr
                prepareBeanFactory(beanFactory);
                // ... 其他步骤
            }
            catch (BeansException ex) {
                // Spring知道如何处理这种异常情况
                if (logger.isWarnEnabled()) {
                    logger.warn("Exception encountered during context
                }
                // 清理已创建的资源
```

```
                destroyBeans();
                cancelRefresh(ex);
                throw ex; // 只有在清理后才抛出
            }
            finally {
                // 清理缓存
                resetCommonCaches();
            }
        }
    }

    // 提供优雅的关闭机制，而不是简单抛异常
    @Override
    public void close() {
        synchronized (this.startupShutdownMonitor) {
            doClose();
            if (this.shutdownHook != null) {
                try {
                    Runtime.getRuntime().removeShutdownHook(this.shutd
                }
                catch (IllegalStateException ex) {
                    // 框架知道这种情况可能发生，不抛给调用者
                    // JVM关闭时可能发生，忽略即可
                }
            }
        }
    }
}

```

异常屏蔽

减少必须处理异常的地方数量的第二种技术是异常屏蔽。使用这种方法，可以在系统的较低级别上检测和处理异常情况，因此，更高级别的软件无需知道该情况。异常屏蔽在分布式系统中尤其常见。

以Spring框架中 `org.springframework.web.client.RestTemplate` 类举例，doExecute方法蔽了各种IO异常、网络异常，统一转换为RestClientException，使得开发者无须关注法底层的IO、网络相关的异常，只需专注包装后的RestClientException即可。

```
public class RestTemplate extends InterceptingHttpAccessor implements

```

```
    @Override
    public <T> ResponseEntity<T> exchange(String url, HttpMethod metho
            @Nullable HttpEntity<?> requestEntity, Class<T> responseTy
            Object... uriVariables) throws RestClientException {

        RequestCallback requestCallback = httpEntityCallback(requestEn
        ResponseExtractor<ResponseEntity<T>> responseExtractor = respo

        // 底层屏蔽了各种IO异常、网络异常，统一转换为RestClientException
        return nonNull(execute(url, method, requestCallback, responseE
    }

    // 异常屏蔽的核心实现
    protected <T> T doExecute(URI url, @Nullable HttpMethod method,
            @Nullable RequestCallback requestCallback,
            @Nullable ResponseExtractor<T> responseExtractor) throws R

        ClientHttpResponse response = null;
        try {
            ClientHttpRequest request = createRequest(url, method);
            if (requestCallback != null) {
                requestCallback.doWithRequest(request);
            }
            response = request.execute();
            handleResponse(url, method, response);
            return (responseExtractor != null ? responseExtractor.extr
        }
        catch (IOException ex) {
            // 屏蔽底层IO异常，转换为业务异常
            String resource = url.toString();
            String query = url.getRawQuery();
            resource = (query != null ? resource.substring(0, resource
            throw new ResourceAccessException("I/O error on " + method
                    " request for \"" + resource + "\": " + ex.getMess
        }
        finally {
            if (response != null) {
                response.close();
            }
        }
    }
}

```

异常聚集

减少与异常相关的复杂性的第三种技术是异常聚合。异常聚合的思想是用一个代码段处理许多异常。与其为多个单独的异常编写不同的处理程序，不如用一个处理程序将它们全部处理在一个地方。

此示例说明了用于异常处理的通用设计模式。如果系统处理一系列请求，则定义一个异常以中止当前请求，清除系统状态并继续下一个请求非常有用。异常被捕获在系统请求处理循环顶部附近的单个位置。在处理中止请求的任何时候都可以抛出该异常。可以为不同的条件定义异常的不同子类。应该将这种类型的异常与对整个系统致命的异常区分开来。

如果异常在处理之前在堆栈中传播了多个级别，则异常聚集最有效。这样可以在同一位置处理更多方法的更多异常。这与异常屏蔽相反：如果使用低级方法处理异常，则屏蔽通常效果最好。对于屏蔽，低级方法通常是许多其他方法使用的库方法，因此，允许传播异常会增加处理该异常的位置数。掩码和聚合的相似之处在于，这两种方法都将异常处理程序置于可以捕获最多异常的位置，从而消除了许多本来需要创建的处理程序。

以Spring MVC中的 `DefaultHandlerExceptionResolver` 可以看到异常聚合理念的具体实现，`doResolveException` 方法在一个地方处理统一各种下层抛出的异常并针对性地处理。

```
1  public class HandlerExceptionResolverComposite implements HandlerExcep
2
3      private List<HandlerExceptionResolver> resolvers;
4
5      @Override
6      @Nullable
7      public ModelAndView resolveException(HttpServletRequest request,
8              HttpServletResponse response,
9              @Nullable Object handler, Exception ex) {
10
11         if (this.resolvers != null) {
12             // 异常聚合：遍历所有解析器来处理不同类型的异常
13             for (HandlerExceptionResolver handlerExceptionResolver :
14                 ModelAndView mav = handlerExceptionResolver.resolveEx
15                     request, response, handler, ex);
16                 if (mav != null) {
17                     return mav;
18                 }
19             }
20         }
```

```
        return null;
    }
}

// 具体的异常解析器 - DefaultHandlerExceptionResolver
public class DefaultHandlerExceptionResolver extends AbstractHandlerEx

    @Override
    @Nullable
    protected ModelAndView doResolveException(HttpServletRequest reque
            HttpServletResponse response,
            @Nullable Object handler, Exception ex) {

        try {
            // 异常聚合：一个方法处理多种不同的异常类型
            if (ex instanceof HttpRequestMethodNotSupportedException)
                return handleHttpRequestMethodNotSupported(
                    (HttpRequestMethodNotSupportedException) ex, reque
            }
            else if (ex instanceof HttpMediaTypeNotSupportedException
                return handleHttpMediaTypeNotSupported(
                    (HttpMediaTypeNotSupportedException) ex, request,
            }
            else if (ex instanceof HttpMediaTypeNotAcceptableExceptior
                return handleHttpMediaTypeNotAcceptable(
                    (HttpMediaTypeNotAcceptableException) ex, request
            }
            else if (ex instanceof MissingPathVariableException) {
                return handleMissingPathVariable(
                    (MissingPathVariableException) ex, request, respor
            }
            else if (ex instanceof MissingServletRequestParameterExcep
                return handleMissingServletRequestParameter(
                    (MissingServletRequestParameterException) ex, requ
            }
            else if (ex instanceof ServletRequestBindingException) {
                return handleServletRequestBindingException(
                    (ServletRequestBindingException) ex, request, resp
            }
            else if (ex instanceof ConversionNotSupportedException) {
                return handleConversionNotSupported(
                    (ConversionNotSupportedException) ex, request, res
            }
            else if (ex instanceof TypeMismatchException) {
                return handleTypeMismatch(
                    (TypeMismatchException) ex, request, response, har
```

```
            }
            else if (ex instanceof HttpMessageNotReadableException) {
                return handleHttpMessageNotReadable(
                    (HttpMessageNotReadableException) ex, request, res
            }
            else if (ex instanceof HttpMessageNotWritableException) {
                return handleHttpMessageNotWritable(
                    (HttpMessageNotWritableException) ex, request, res
            }
            else if (ex instanceof MethodArgumentNotValidException) {
                return handleMethodArgumentNotValidException(
                    (MethodArgumentNotValidException) ex, request, res
            }
            else if (ex instanceof MissingServletRequestPartException
                return handleMissingServletRequestPartException(
                    (MissingServletRequestPartException) ex, request,
            }
            else if (ex instanceof BindException) {
                return handleBindException((BindException) ex, reques
            }
            else if (ex instanceof NoHandlerFoundException) {
                return handleNoHandlerFoundException(
                    (NoHandlerFoundException) ex, request, response, h
            }
            else if (ex instanceof AsyncRequestTimeoutException) {
                return handleAsyncRequestTimeoutException(
                    (AsyncRequestTimeoutException) ex, request, respon
            }
        }
        catch (Exception handlerEx) {
            if (logger.isWarnEnabled()) {
                logger.warn("Failure while trying to resolve exceptio
            }
        }
        return null;
    }
}

```

该崩溃就奔溃

减少与异常处理相关的复杂性的第四种技术是使应用程序崩溃。在大多数应用程序中，有些错误是不值得尝试的。通常，这些错误很难或不可能处理，而且很少发生。针对这些错误的最简

单的操作是打印诊断信息，然后中止应用程序。

以Spring中的最经典的入口 `SpringApplication` 类举例， 看下它在启动失败时，是如何处理的。核心代码在 `run` 方法第一个try catch内的 `handleRunFailure` 方法，在遇到致命错误时，启动会导致应用崩溃。

```
public class SpringApplication {

    public static ConfigurableApplicationContext run(Class<?> primaryS
        return run(new Class<?>[] { primarySource }, args);
    }

    public static ConfigurableApplicationContext run(Class<?>[] prima
        return new SpringApplication(primarySources).run(args);
    }

    public ConfigurableApplicationContext run(String... args) {
        StopWatch stopWatch = new StopWatch();
        stopWatch.start();
        ConfigurableApplicationContext context = null;
        Collection<SpringBootExceptionReporter> exceptionReporters = r
        configureHeadlessProperty();
        SpringApplicationRunListeners listeners = getRunListeners(args
        listeners.starting();

        try {
            ApplicationArguments applicationArguments = new DefaultApp
            ConfigurableEnvironment environment = prepareEnvironment(
            configureIgnoreBeanInfo(environment);
            Banner printedBanner = printBanner(environment);
            context = createApplicationContext();
            exceptionReporters = getSpringFactoriesInstances(SpringBoo
                    new Class[] { ConfigurableApplicationContext.class
            prepareContext(context, environment, listeners, applicatio
            refreshContext(context);
            afterRefresh(context, applicationArguments);
            stopWatch.stop();

            if (this.logStartupInfo) {
                new StartupInfoLogger(this.mainApplicationClass).logSt
            }
            listeners.started(context);
            callRunners(context, applicationArguments);
        }
        catch (Throwable ex) {
```

```
            // 关键：不可恢复的启动错误直接崩溃应用
            handleRunFailure(context, ex, exceptionReporters, listene
            throw new IllegalStateException(ex);  // 直接抛出导致应用崩溃
        }

        try {
            listeners.running(context);
        }
        catch (Throwable ex) {
            // 运行时严重错误也直接崩溃
            handleRunFailure(context, ex, exceptionReporters, null);
            throw new IllegalStateException(ex);
        }
        return context;
    }

    private void handleRunFailure(ConfigurableApplicationContext conte
            Collection<SpringBootExceptionReporter> exceptionReporters
            SpringApplicationRunListeners listeners) {
        try {
            try {
                handleExitCode(context, exception);
                if (listeners != null) {
                    listeners.failed(context, exception);
                }
            }
            finally {
                // 打印诊断信息
                reportFailure(exceptionReporters, exception);
                if (context != null) {
                    context.close();
                }
            }
        }
        catch (Exception ex) {
            logger.warn("Unable to close ApplicationContext", ex);
        }
        // 注意：这里不捕获异常，让应用自然崩溃
        ReflectionUtils.rethrowRuntimeException(exception);
    }

    private void reportFailure(Collection<SpringBootExceptionReporter>
            Throwable failure) {
        try {
            // 打印详细的诊断信息
            for (SpringBootExceptionReporter reporter : exceptionRepo
```

```
                if (reporter.reportException(failure)) {
                    registerLoggedException(failure);
                    return;
                }
            }
        }
        catch (Throwable ex) {
            // 连诊断信息都无法打印时，记录基本错误
            logger.warn("Error reporting exception", ex);
        }

        // 确保错误被记录
        if (logger.isErrorEnabled()) {
            logger.error("Application run failed", failure);
        }
    }
}

```

在特定错误上崩溃是否可以接受取决于应用程序。对于复制的存储系统，不适合因I/O错误而中止。相反，系统必须使用复制的数据来恢复丢失的任何信息。恢复机制将给程序增加相当大的复杂性，但是恢复丢失的数据是系统为用户提供的价值的重要组成部分。



软件设计并不容易，尤其是在复杂的框架层代码。往往来说，对模块或系统的第一版设计不太可能是最佳的设计。如果为每个主要设计决策考虑多个选项，最终将获得更好的结果：设计两次。

尝试选择根本不同的方法，或者说做技术选型；这样将学到更多知识。极端一点说，即使当前场景确定只有一种合理的方法，你都应该尝试考虑第二种设计。考虑该设计的弱点并将它们与其他设计的特征进行对比将很有启发性。在对备选方案进行粗略设计之后，列出每个方案的优缺点。

比较了备选设计之后，如果主设计方案的确更佳，那可以佐证主设计的确最佳。或者我们可能在过程中发现，将多个方案的部分功能组合到一个比任一原始设计都要好。

有些人可能会顾虑额外设计带来的时间成本。但是进行两次设计不需要花费很多额外的时间。对于较小的模块，可能一两个小时就能搞定，这和实现该方案的时间（往往是几天或者一周以

上）相比实现不值一提。最初的设计方案可能会导致明显更好的设计方案，这比早期多投入的时间划算的多。对于较大的模块，我们将花费更多的时间进行初始设计探索，但是实现起来也将花费更长的时间，并且更好的设计带来的收益也会更高。



迭代型重构（小步快跑）

大型软件系统是无一例外都是通过一系列演化发展而来的，每个阶段都添加了新功能并修改了现有模块。这意味着系统的设计在不断发展。一开始就不可能为系统设计正确的设计。一个成熟的系统的设计更多地取决于系统演化过程中所做的更改，而不是一开始所的初始设想。

在战术编程中，主要目标是使某些功能快速工作，即使这会导致额外的复杂性；**在战略编程中**，最重要的目标是进行出色的系统设计。战术方法很快导致系统设计混乱。如果我们想要一个易于维护和增强的系统，那么“能工作”是远远不够的。我们必须优先考虑设计并进行战略思考。

不幸的是，当开发者进入现有代码以进行更改时，他们通常不会从战略角度进行思考。一个典型的心态是“我能做出我需要做的最小的改变是什么？”有时，开发者证明这是合理的，因为他们对修改的代码不满意。他们担心较大的更改会带来更大的引入新错误的风险。但是，这导致了战术编程。这些最小的变化中的每一个都会引入一些特殊情况、依赖或其他形式的复杂性。结果，系统设计变得更糟，并且问题随着系统发展的每个步骤而累积。

如果要维护系统的简洁设计，则在修改现有代码时必须采取战略性方法。理想情况下，当我们完成每次更改时，如果我们从一开始就考虑到更改就设计了系统，那么系统将具有它应该具有的结构。为了实现此目标，我们必须抵制诱惑以快速解决问题。相反，请根据所需的更改来考虑当前的系统设计是否仍然是最佳的。如果不是，请重构系统，以便最终获得最佳设计。通过这种方法，每次修改都会改善系统设计。

如果我们花费一些额外的时间来重构和改善系统设计，我们将得到一个更干净的系统。最终将加快开发速度，我们将回收在重构方面投入的时间和精力。即使我们的特定更改不需要重构，我们仍然应该注意在代码中可以修复的设计缺陷。每当我们修改任何代码时，都尝试在该过程中至少找到一点方法来改进系统设计。如果我们没有使设计更好，则可能会使它变得更糟。

有些人可能会说，业务的需求很急，实在抽不出时间来重构。比如某个功能如果要重构后上线需要二个月，但是按照现在的逻辑修修补补只要两个小时，采取快速而肮脏的方法很值。又或

者说，如果重构系统会造成许多团队其他成员的模块也需要修改，上下游会不兼容，重构不切实际。

的确在大多数时候，业务需求总是很急，留给我们完善的设计和实现时间并不多。但是即便如此，我们还是应该尽可能抵制这些妥协。可以问问自己：“考虑到目前的时间，这是不是我能做到的最干净的系统设计？也许有一种替代方法几乎可以像2个月的重构一样干净，但是可以在几天内完成？或者，如果我们现在负担不起大型重构，请让我们的老板为我们分配时间，或者找一个业务需求相对不紧急的时候。

总体来说，每个开发团队都应计划将其全部工作的一小部分用于清理和重构。从长远来看，这很值得。



一致性是降低系统复杂性并使其行为更明显的强大工具。如果系统是一致的，则意味着相似的事情以相似的方式完成，而不同的事情则以不同的方式完成。一致性会产生认知影响力：一旦我们了解某个地方的工作方式，就可以使用该知识立即了解其他使用相同方法的地方。如果系统的实施方式不一致，则开发者必须分别了解每种情况。这将花费更多时间。

一致性减少了错误。如果系统不一致，则实际上两种情况可能不同，但两种情况可能看起来相同。开发者可能会看到一个看起来很熟悉的模式，并根据以前对该模式的遭遇做出错误的假设。另一方面，如果系统是一致的，则基于熟悉情况的假设将是安全的。一致性允许开发者以更少的错误来更快地工作。

以下是系统一致性的几种体现：

- 编码样式。如今，开发团队通常拥有样式指南，这些样式指南将程序结构限制在编译器所强制执行的规则之外。现代风格指南解决了一系列问题，例如缩进，大括号放置，声明顺序，命名，注释以及对认为危险的语言功能的限制。样式指南使代码更易于阅读，并且可以减少某些类型的错误。
- 接口。具有多个实现的接口是一致性的另一个示例。一旦了解接口的一种实现，其他任何实现都将变得更易于理解，因为我们已经知道它将必须提供的功能。
- 设计模式。设计模式是某些常见问题的普遍接受的解决方案，例如用于用户接口设计的模型视图控制器方法。如果我们可以使用现有的设计模式来解决问题，则实现会更快地进行，更有可能起作用，并且我们的代码对阅读代码的人来说也会更明显。

一致性很难维护，尤其是当许多人长时间从事一个项目时。一组人可能不了解另一组中建立的约定。新来的同学不了解规则，因此他们无意间违反了约定并创建了与现有约定冲突的新约定。以下是建立和保持一致性的一些技巧：

- 文档。创建一份文档，列出最主要的重要约定。比如编码风格指南（阿里巴巴有Java编码规范）。
- 执行。即使有好的文档，开发者也很难记住所有约定。实施约定的最佳方法是编写一个检查违规的自动化工具，并确保代码通过检查程序。

一致性是投资心态的另一个例子。确保一致性的工作将需要一些额外的工作：确定约定，创建自动检查程序，寻找类似情况以模仿新代码，以及进行代码审查以教育团队。这项投资的回报使我们的代码将更加明显。开发者将能够更快，更准确地了解代码的行为，这将使他们能够以更少的错误来更快地工作。



如果代码很明显，则意味着某人可以不假思索地快速阅读该代码，并且他们对代码的行为或含义的最初猜测将是正确的。如果代码很明显，那么阅读代码的人就不需要花费很多时间或精力来收集他们使用代码所需的所有信息。如果代码不明显，那么阅读代码的人必须花费大量时间和精力来理解它。这不仅降低了它们的效率，而且还增加了误解和错误的可能性。明显的代码比不明显的代码需要更少的注释。

阅读代码的人的想法是“显而易见”：注意到别人的代码不明显比发现自己的代码有问题要容易得多。因此，确定代码是否显而易见的最佳方法是通过代码审查。如果有人在阅读我们的代码时说它并不明显，那么无论我们看起来多么清晰，它也不是显而易见。通过尝试理解什么使代码变得不明显，我们将学习如何在将来编写更好的代码。

软件应设计为易于阅读而不是易于编写。通用容器对于编写代码的人来说是很方便的，但是它们会使随后的所有阅读代码的人感到困惑。对于编写代码的人来说，花一些额外的时间来定义特定的容器结构是更好的选择，以便使生成的代码更加明显。

直观的技巧

- 选择好名字：精确而有意义的名称可以阐明代码的行为，并减少对文档的需求。如果名称含糊不清或含糊不清，那么阅读代码的人将通读代码以推论命名实体的含义；这既费时又容易出错。

- 一致性：如果总是以相似的方式完成相似的事情，那么阅读代码的人可以识别出他们以前所见过的模式，并立即得出结论，而无需详细分析代码

以下是使代码更明显的其他一些通用技术：

- 使用空白。代码格式化的方式会影响其理解的容易程度。（代码）
- 注释。有时无法避免非显而易见的代码。发生这种情况时，重要的是使用注释来提供缺少的信息以进行补偿。（代码）

不直观的案例

- 事件驱动编程。在事件驱动编程中，应用程序对外部事件做出响应。应用程序的其他部分通过在事件发生时要求事件模块调用给定的函数或方法来注册对某些事件的兴趣。倒不是说事件驱动编程这种思想不好，而是说这种思想写出的代码对阅读代码的人的确增加了模糊性。为了弥补这种模糊性，可以为每个处理程序函数使用接口注释，以明确说服何时调用该函数。
- 通用容器。许多语言提供了用于将两个或多个结构到一个对象中的通用类，例如Java中的Pair。这些类很诱人，常见的用途是从一个方法返回多个值。不幸的是，通用容器导致代码不清晰，因为分组后的元素的通用名称模糊了它们的含义。JDK17中Java提供了新的record类型，合理的使用record可以有效提高代码的可读性。
- 反直觉。如果代码符合阅读代码的人期望的惯例，那么它是最明显的。如果没有，那么为该行为增加注释很重要，以免使阅读代码的人感到困惑。比如fastJson框架是通过执行类中的getXXX方法实现对象构造的，如果在类中有非字段相关的getXXX也会被执行（曾经触发过很多故障）。



到目前为止，关于软件设计的讨论都集中在复杂性上。目标是使软件尽可能简单易懂。但是，如果我们需要高性能的系统实现，该怎么办？性能方面的考量会如何影响设计过程？

要解决的第一个问题是“在正常的开发过程中应该为性能花多少心思？”。因为优化性能需要不少的时间成本，同时也增加了复杂性。假设如果每条语句都需要尝试优化以获得最高的性能，那么它不仅会大幅度减慢开发速度，同时许多不必要的复杂性。此外，许多“优化”实际上对性能并没有帮助。但是如果完全不考虑性能问题，则可能导致整个代码的效率低下，最终系统比预想的速度慢5—10倍。（总不能ToB或者ToC的接口RT在秒级）

如果提高效率的唯一方法是增加复杂性，那么选择起来就会比较困难。如果更高性能的设计仅增加了少量复杂性，并且复杂性是隐藏的，那么可以放心地选择性能更优的方案。但是如果更

高性能设计需要增加很多复杂性，那么最好是先从更简单的实现开始，然后在性能出现瓶颈时进行优化。但是，如果我们有明确的需求表明性能在特定情况下很重要（比如我曾经做过一个需求必须满足所有投放逻辑包括外调在5ms内处理），那么最好一开始就选择性能更好的方法。

如果真的决定了需要优化性能，那么先好好测量系统的现有表现。这有两个目的。

- 首先，这些测量将确定对性能影响最大的地方。仅仅测量系统性能是不够的。虽然我们能知道系统速度太慢，但不知道原因。我们还需要进行更深入的测量，详细确定影响整体性能的核心因素。
- 其次是提供基线，以便在优化后重新测量性能，以确保性能得到实际改善。如果这些更改没有显著提高性能，则保持优化带来的复杂性毫无意义。



全文只讲一件事：复杂性。控制复杂性是软件设计的核心挑战。它让系统难以构建、难以维护，也常常拖慢运行效率。

虽然本文中的建议在实操时会带来一定的代价：它们会在项目早期增加额外工作量；如果我们不习惯从设计角度思考，学习这些技巧会让我们一时变慢。但是如果我们的唯一目标是尽快把当前功能跑起来，设计思考看起来就像是在“挡路”，妨碍我们达成眼前目标。

但如果我们把良好设计当成重要目标，这些想法会让编程更有趣。设计是一道迷人的题：用最简单的结构解决特定问题。探索不同方案本身就很有意思，而当我们找到一个既简单又有力的解法时，会有一种难以替代的满足感。简单、清晰的系统本身就是一种美。

更重要的是，这种投入很快回本。我们会不断复用在项目初期精心定义的模块和边界，从而节省大量时间。

成为优秀的设计者的回报在于：我们能把更多时间花在设计与抽象上；而糟糕的设计者则把大部分时间耗在修复复杂、脆弱代码中的各种问题上。



- 《软件设计的哲学》
- 《架构整洁之道》
- 《Google软件工程》

- 《OnJava中文版》
- 《凤凰架构》
- 《软件架构设计：大型网站技术架构与业务架构融合之道》
- 《系统架构：复杂系统的产品设计与开发》
- 《架构师修炼之道》



本文作者思诚，来自淘天集团营销&交易技术团队。本团队承担淘天电商全链路交易技术攻坚，致力于通过技术创新推动业务增长与用户体验升级。过去一年主导了多个高价值项目，包括：支撑618、双11、春晚等亿级流量洪峰、构建业界领先的全网价格力体系、承接淘宝全面接入微信支付、搭建集团最大的AI创新平台-ideaLAB，支撑淘宝秒杀等创新业务的高速增长。

拓展阅读

3DXR技术 | 终端技术 | 音视频技术

服务端技术 | 技术质量 | 数据算法

大淘宝技术

大淘宝技术官方账号

967篇原创内容

公众号

阅读 7436

留言

写留言

还没有留言，来写首评